



Maintaining Quality When Moving Fast

2024



Today's topics

- Introductions: Jussi and Unity
- Continuous delivery and testing

BREAK?

- Expanding the testing horizon
- Discussion on selected topics

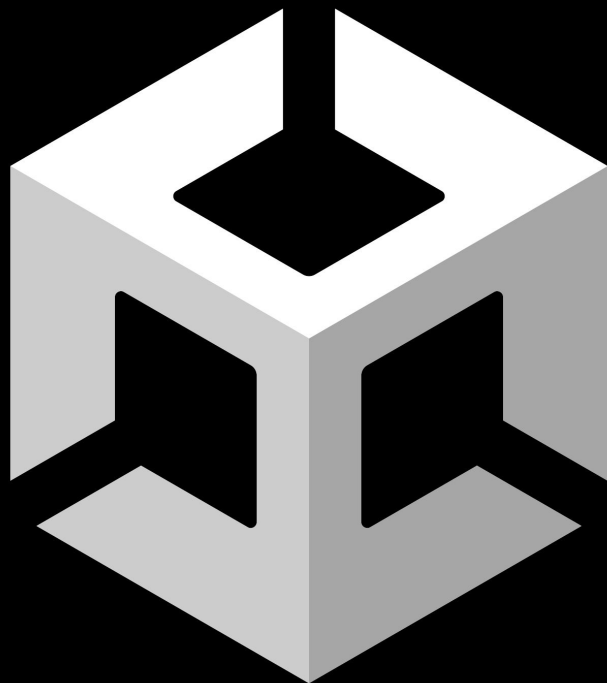


Nice to meet you!



Jussi Kuosa

Senior Software Engineer @Unity

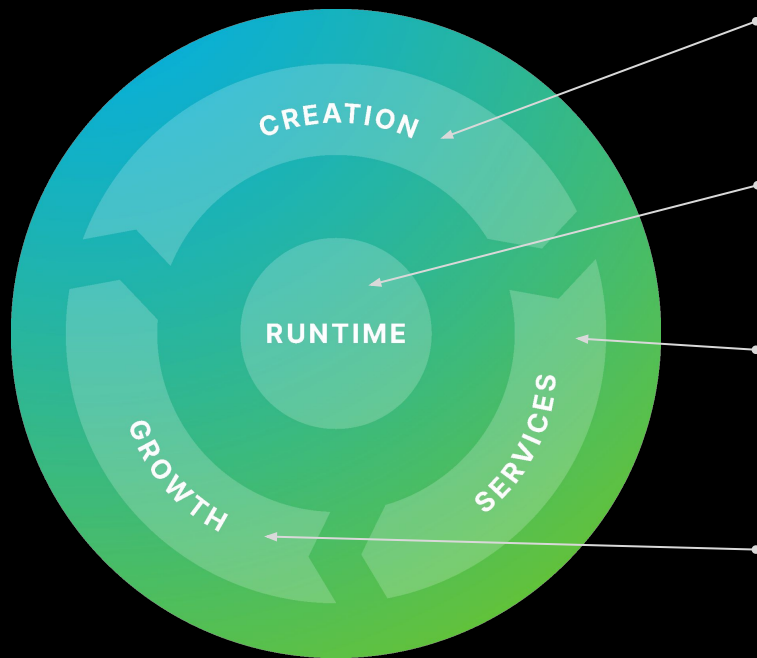




**“We believe the world is a
better place with more
creators in it.”**



SOLUTIONS AT EVERY STEP FOR CREATORS



UNITY ENGINE AND EDITOR

The Unity Editor is highly customizable and provides a visual interface for designing levels, creating game logic, and managing assets. The Unity Engine empowers creators to craft games, apps, or immersive experiences, featuring multiplatform support, with AI integration options.

UNITY RUNTIME

Unity Runtime is our proprietary software that enables games to run across 20+ platforms seamlessly. It processes game logic, renders graphics, manages inputs, and supports physics, animations, and networking directly on devices.

UNITY CLOUD

Unity Cloud provides cloud-based and connected services for developers to efficiently develop, launch, and scale games. It facilitates collaboration on real-time 3D (RT3D) projects, automates builds, tests across environments, and supports multiplayer features on over 20+ platforms.

UNITY GROW

Unity Grow offers tools for monetizing and acquiring users, featuring LevelPlay, ad networks, TapJoy Offerwall, and Supersonic for scaling mobile games into successful businesses.



TIME GHOST

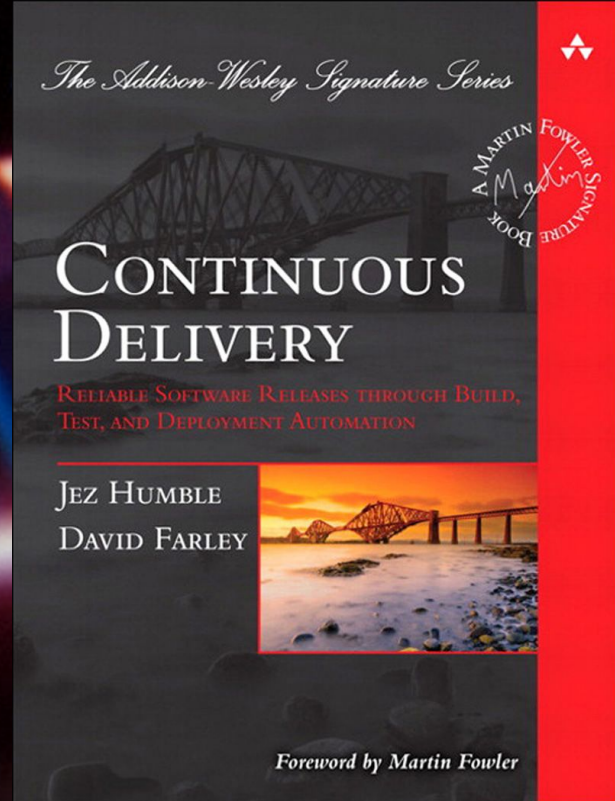


NOKIA 3310

the Chuck Norris of mobile phones



Moving Fast with Continuous Delivery



Drivers for continuous delivery

Ultimately, only the customer is able to tell the value and quality of your service

- We want to get product increments to customers as soon as possible
 - Fast feedback and time-to-market, TTM

Large releases carry a lot of inherent risk

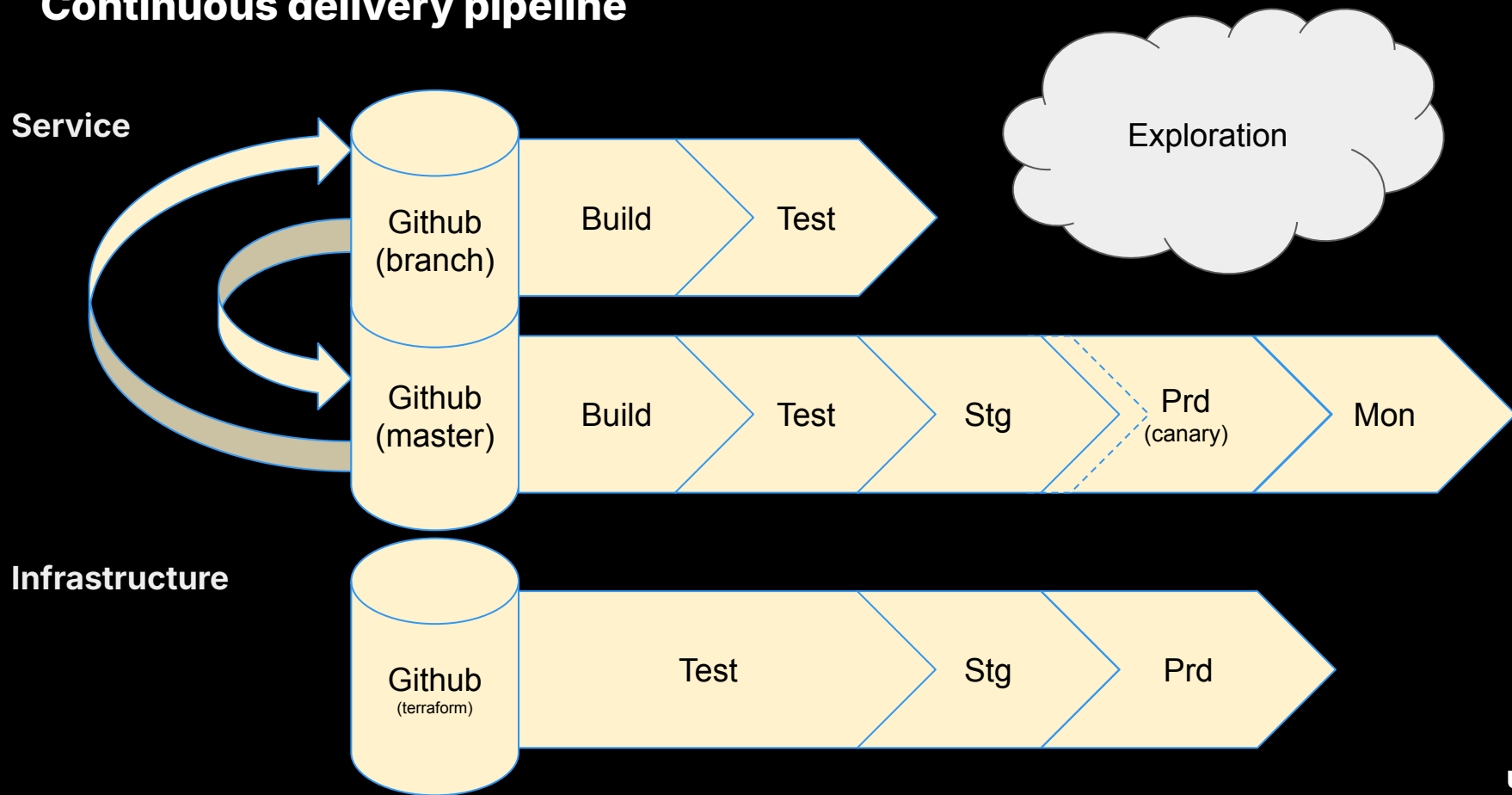
- Features end up sitting on the to-be-shipped shelf for long time
- All-or-nothing approach, long turnaround time if fixes needed
- Too often, development slips and release date is carved in stone

As delivery frequency increases, less time for massive release test phase

- Fully automated build and delivery pipeline for consistent releases
- Need a fast (=automated) regression test suite
- Currently the slowest component takes about 30 minutes to deploy fully



Continuous delivery pipeline





DORA Metrics

Deployment Frequency

How often an organization successfully releases to production

Lead Time for Changes

The amount of time it takes a commit to get into production

Change Failure Rate

The percentage of deployments causing a failure in production

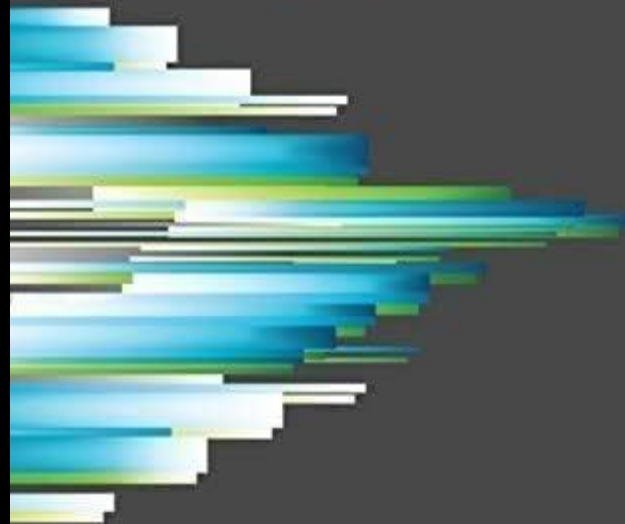
Time to Restore Service

How long it takes an organization to recover from a failure in production

+Reliability

THE SCIENCE OF DEVOPS ACCELERATE

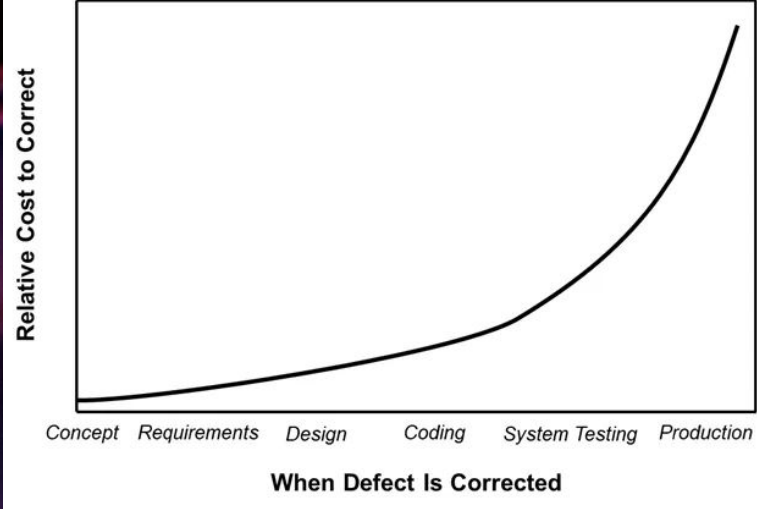
Building and Scaling High Performing
Technology Organizations



Nicole Forsgren, PhD
Jez Humble *and* Gene Kim



Continuous Testing



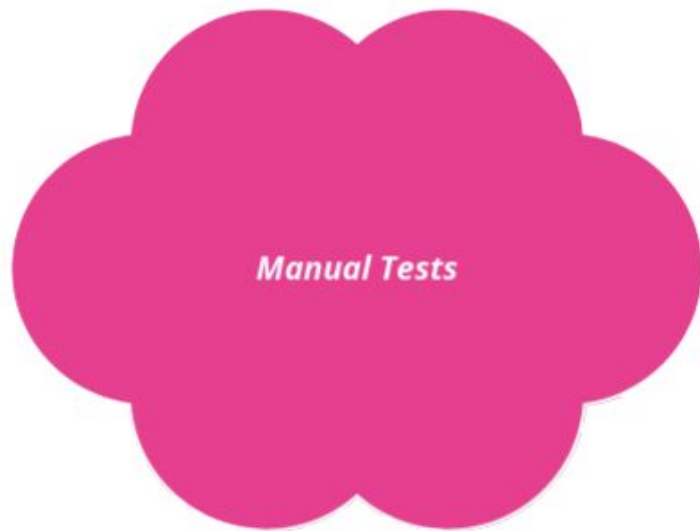


**"Teams own the quality
of their services."**



Quality Fundamentals

- Everything in code repository
- Pull requests and code reviews
- CI/CD pipeline
- Linters
- Unit tests
- Manual testing

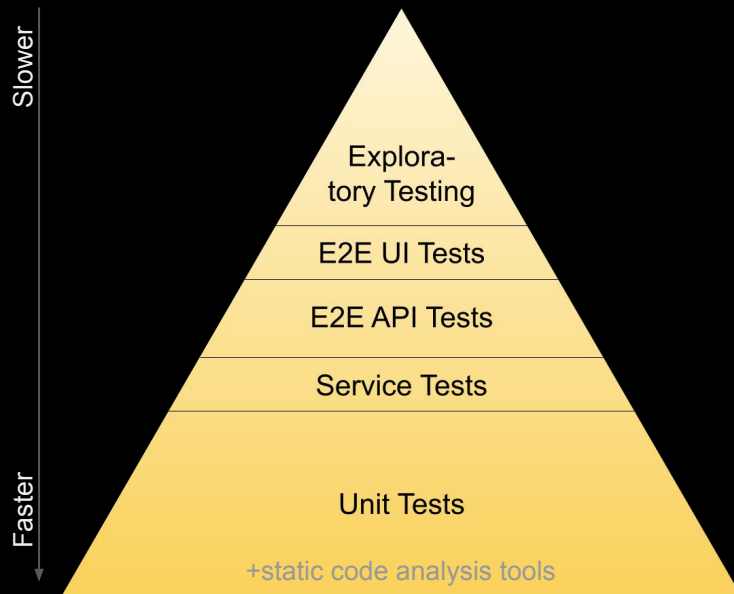


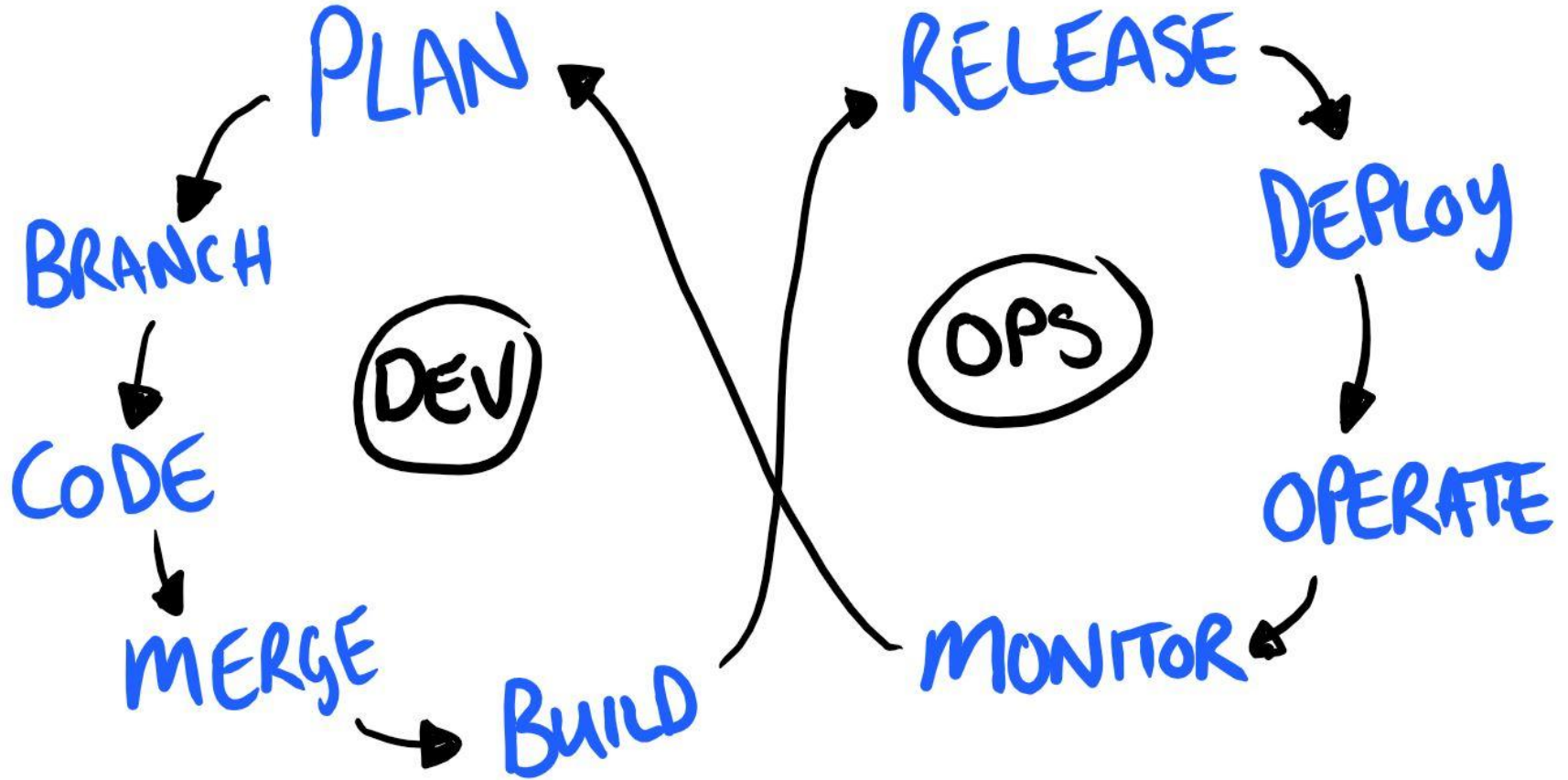


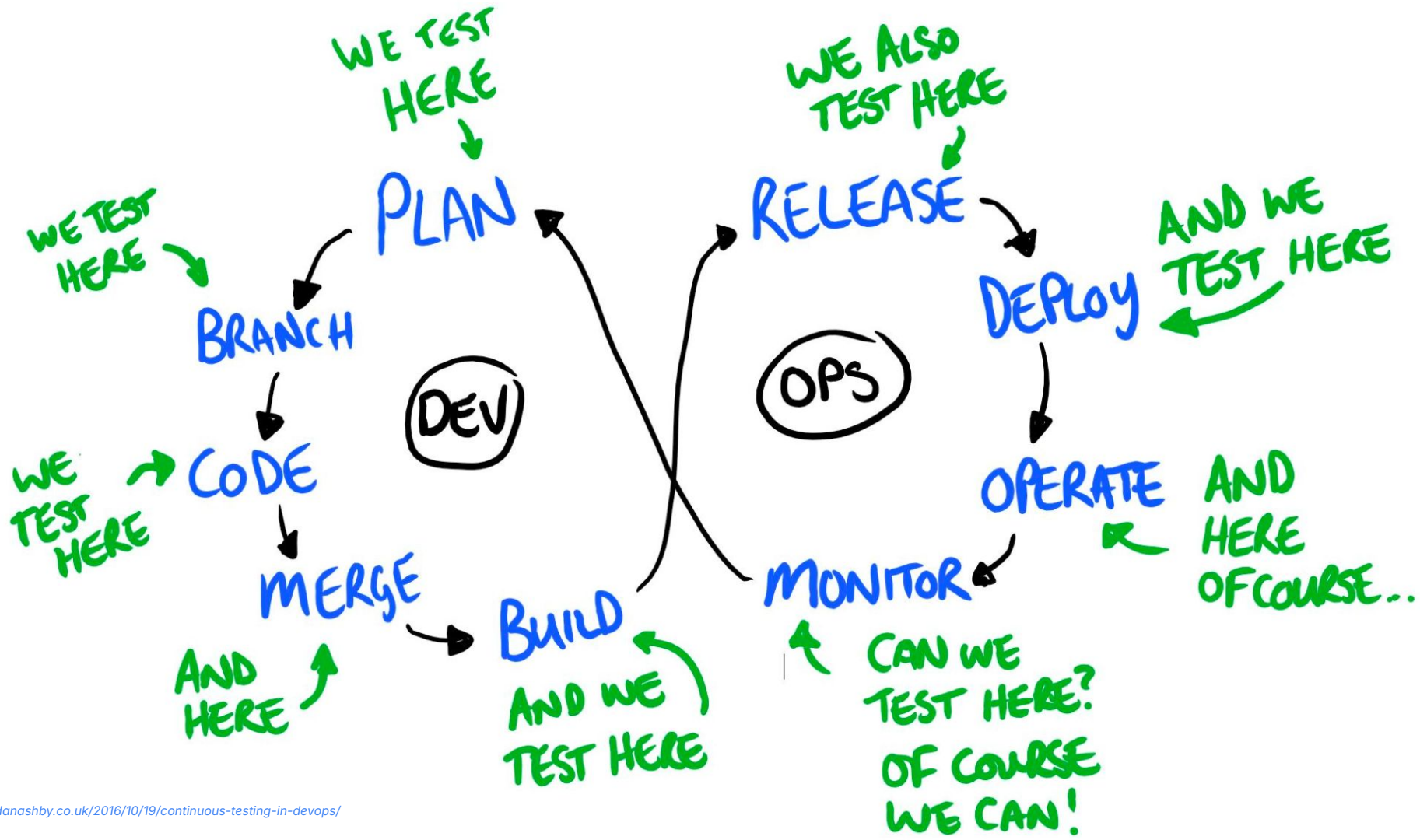
Quality Fundamentals

- Everything in code repository
 - *infrastructure as code* as well
- Pull requests and code reviews
- CI/CD pipeline
- *Formatters*
- Linters
- Unit tests with known *coverage*
- *API/integration tests*
- *UI/system tests*
- *Exploratory testing*

(+maybe load/fuzz/lln/i18n/a11y)









**"As a software engineer
I've noticed that one of our
biggest challenges isn't the
code
but understanding each other."**

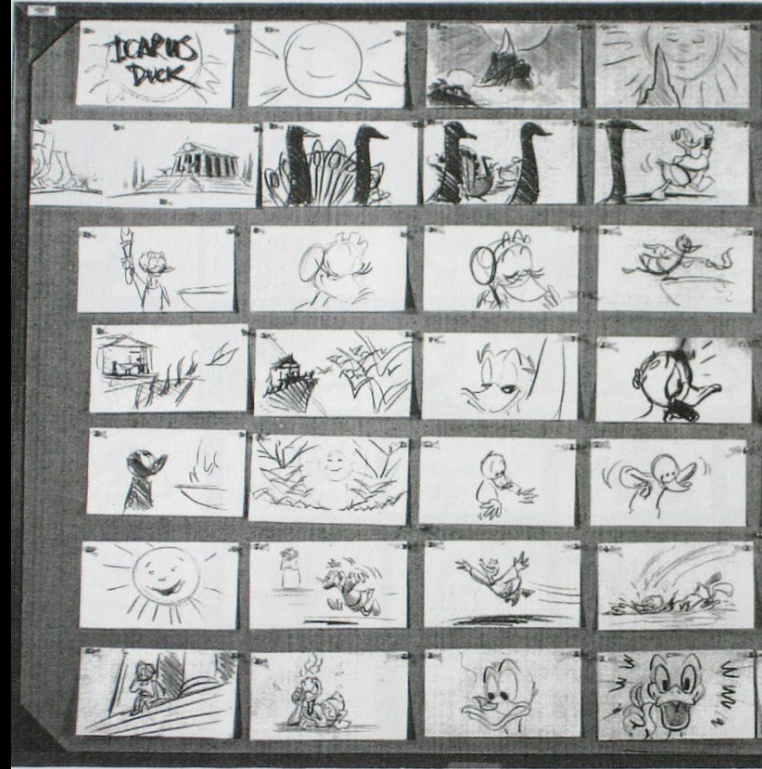
Łukasz Roszak



Shift-left testing

As Unity grew, we started using feature design documents to tie together work from numerous teams:

- Writing down the intent helps clarify the scope (to fight nodding-syndrome)
- Facilitates **discussion** across teams across multiple time zones
- Extract **customer journeys**
- Great opportunity to catch **idea bugs**
- Fast comment-correction cycle

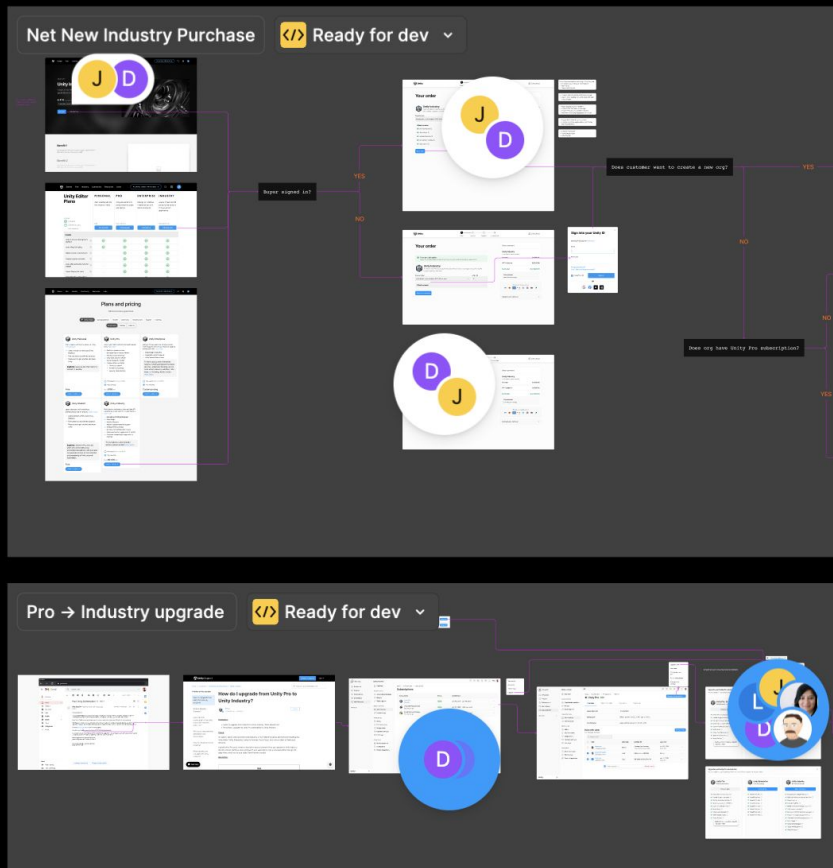


Validation - Are we building the right product?



- Splitting bigger features to smaller increments (milestones)
- Validating customer journeys on UX wireframes
- Prototyping (achieve golden flow)
 - Company hackweeks
 - Cross-team workshops

The goal is to reduce risk as early as possible by removing ambiguity and misalignment and forming a clear done/acceptance criteria.





Shift-right testing

Testing in production

- Observability
 - Logs, metrics, alerts, tracing
- Feature flags
- A/B testing
- Canary releases

Consequences

- Incidents

Verification - Are we building the product right?



PRE-PRODUCTION

- UNIT TESTS
- FUNCTIONAL TESTS
- COMPONENT TESTS
- FUZZ TESTS
- STATIC ANALYSIS
- PROPERTY BASED TESTS
- COVERAGE TESTS
- BENCHMARK TESTS
- REGRESSION TESTS
- CONTRACT TESTS
- LINT TESTS
- ACCEPTANCE TESTS
- MUTATION TESTS
- SMOKE TESTS
- UI/UX TESTS
- USABILITY TESTS
- PENETRATION TESTS
- THREAT MODELLING

TESTING IN PRODUCTION

← DEPLOY → ← RELEASE → ← POST RELEASE →

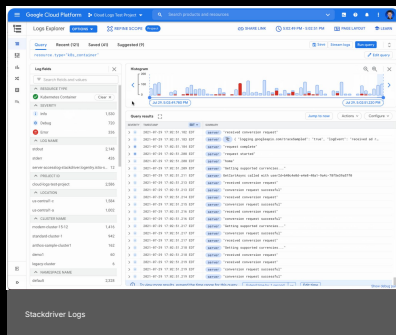
- INTEGRATION TESTS
- TAP COMPARE
- LOAD TESTS
- SHADOWING
- CONFIG TESTS

- CANARYING
- MONITORING
- TRAFFIC SHAPING
- FEATURE FLAGGING
- EXCEPTION TRACKING

- TEEING
- PROFILING
- LOGS/EVENTS
- CHAOS TESTING
- MONITORING
- A/B TESTS
- TRACING
- DYNAMIC EXPLORATION
- REAL USER MONITORING
- AUDITING
- ONCALL EXPERIENCE



Observability



Logs

Silent by default

Add logs when needed



Metrics

Actionable metrics

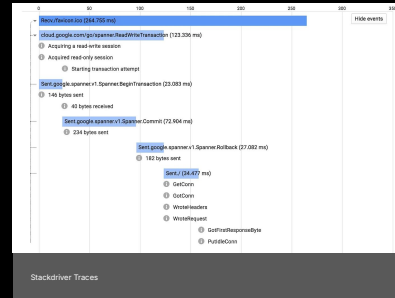
Avoid vanity metrics



Alerts

24/7 rotation

Runbooks for known scenarios



Traces

Drill-down to failing component

Combined view with logs



Feature Flags

Distributed Feature Flags

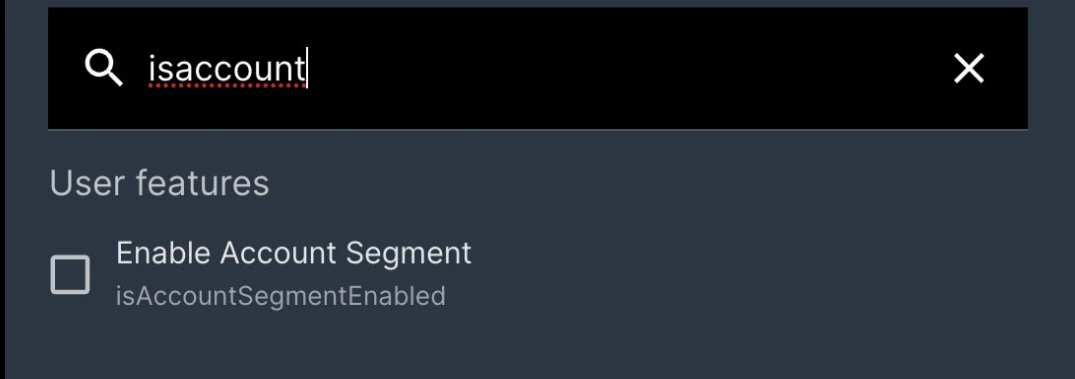
- Both front and backends
- Work across microservices

Enable gradual roll-out

- Enable first to beta-customers
- Remove after successful roll-out

Enable graceful degradation

- Disable the problematic feature, and run other features normally



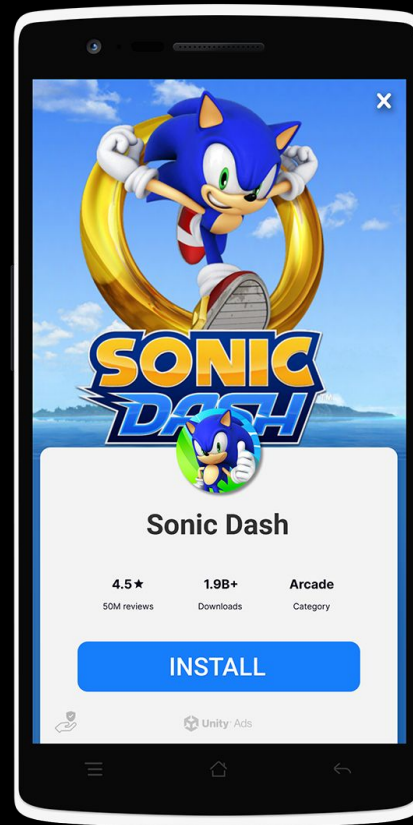
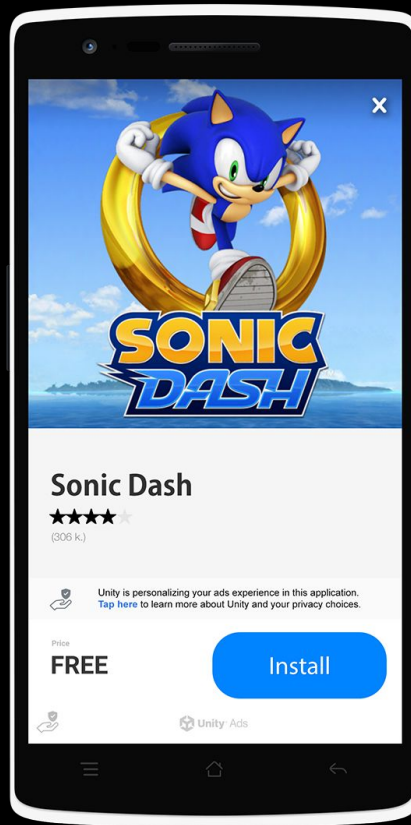
A/B Testing

Design an experiment with control and variant

Show control and variant to different groups

- Duration must be long enough for statistical significance (~1week)

Compare analytics between alternatives for desired outcome





Canary Releases

Split production to 2 (or more) segments

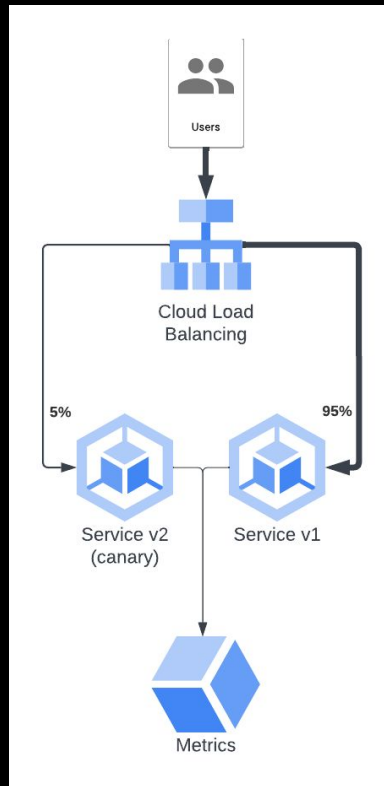
First, deploy new version to small canary area

Observe metrics for expected impact

- Less errors (bug fix)
- Reduced latency (optimization)

Roll out to full production

- OR, roll back with minimal impact





Incidents

Sometimes, we fail...

- Alerts help trigger incidents

Incidents have highest priority

- Metrics and logs allow fast pinpointing
- Tracing helps with layered systems

Always look for roll back possibility

- Stop new A/B test
- Disable feature if just exposed
- Investigate and fix in peace

Retrospective for continuous improvement



03:05 **OpsBot** APP Incident [#incident-2023-10-27](#) started by [@bipen](#) (edited)

Description

ads brand pods in crash loop backoff during deployment

Priority

P1: Service Degradation

Business unit

monetize

Latest update

This incident is not resolved. I still see CrashLoops on the latest rollout.

Resolved, waiting for postmortem



[16 replies](#) Last reply 12 days ago



Group Assignments

1. 100% unit test coverage - are we done?
 - a. What does it mean?
 - b. What defects might slip?
2. Exploratory testing when moving fast?
 - a. How can exploration be added to the mix?
3. How can the team own the quality?
 - a. What has to be done differently?
 - b. What are the benefits and downsides?
4. Performance



We have an internship program!

careers.unity.com



Thank you

2023



